

JUSTIFICACIÓN DE TECNOLOGÍAS UTILIZADAS

1. NativeScript 9 Core

NativeScript fue asignado al grupo por sorteo, sin embargo la tecnología resultó apropiada para la problemática de gestión de órdenes y entregas. Sus principales ventajas para este proyecto son el acceso directo al 100% de las APIs nativas de Android sin necesidad de código Java o Kotlin adicional, componentes de interfaz nativos reales que garantizan rendimiento cercano al nativo, soporte completo para TypeScript, plugins oficiales para todas las funcionalidades avanzadas requeridas (cámara, notificaciones push, SQLite, imágenes) y licencia Apache 2.0 de código abierto.

La problemática requería notificaciones push, almacenamiento local, integración con API REST, manejo de imágenes y autocompletado de direcciones.

NativeScript cubre todos estos casos con plugins de primera parte sin depender de soluciones de terceros.

2. TypeScript 5.4

Se eligió TypeScript sobre JavaScript porque permite detectar errores de contrato con la API en tiempo de desarrollo antes de ejecutar la aplicación. El sistema de metadatos automáticos de NativeScript genera definiciones de tipos TypeScript para todas las APIs nativas de Android, lo que proporciona autocompletado completo en el editor. Al trabajar varios integrantes en paralelo, los tipos explícitos reducen los errores de integración entre módulos. Es el lenguaje recomendado oficialmente por el equipo de NativeScript para proyectos de mediana y gran escala.

3. @nativescript-community/sqlite

Se utilizó para implementar el almacenamiento local de entregas que permite el funcionamiento offline de la aplicación. Es un plugin de la comunidad NativeScript con soporte activo y API asíncrona basada en promesas compatible con `async/await` de TypeScript. Permite almacenar colecciones estructuradas mediante SQL estándar, a diferencia de `ApplicationSettings` que solo admite pares clave-valor. Funciona con el módulo SQLite nativo de Android sin dependencias adicionales.

4. @nativescript/background-http

Se utilizó para el envío de fotos de perfil de clientes y repartidores al servidor. Es el único plugin oficial de NativeScript que soporta el formato multipart/form-data con archivos adjuntos, necesario para enviar una imagen junto con los datos del formulario en una sola petición HTTP. El módulo Http estándar de NativeScript Core no soporta multipart de forma nativa, por lo que este plugin fue la solución adecuada. Es mantenido directamente por el equipo oficial de NativeScript.

5. @nativescript/camera y @nativescript/imagepicker

Se utilizaron para la captura y selección de fotos de perfil en los formularios de registro de clientes y repartidores. El plugin de cámara provee acceso directo al hardware de la cámara del dispositivo con manejo automático de permisos en Android. El plugin de imagepicker provee acceso a la galería de fotos del dispositivo. Ambos son plugins oficiales del equipo NativeScript con soporte para Android 9 en adelante y devuelven la imagen en un formato directamente compatible con el ecosistema NativeScript.

6. OpenStreetMap Nominatim

Se utilizó como API externa de geocodificación para el autocomplete de direcciones en las pantallas de crear y editar entrega. Se eligió porque es completamente gratuita sin necesidad de API key ni registro, tiene cobertura de Costa Rica con el parámetro countrycodes=cr, devuelve respuestas en formato JSON con un campo display_name listo para mostrar al usuario, y su límite de una petición por segundo se maneja adecuadamente con el debounce de 500ms implementado. El endpoint utilizado es GET <https://nominatim.openstreetmap.org/search>.

7. ApplicationSettings de NativeScript Core

Se utilizó para persistir el token JWT y el rol del usuario entre sesiones de la aplicación. Es una API nativa incluida en @nativescript/core que internamente usa SharedPreferences de Android, por lo que no requiere dependencias adicionales. Es apropiada para almacenar cadenas de texto pequeñas como tokens de autenticación que deben sobrevivir al cierre y reapertura de la aplicación.

8. API de Connectivity de NativeScript Core

Se utilizó para verificar el estado de red del dispositivo antes de realizar llamadas a la API REST o al servicio de geocodificación. Está incluida en @nativescript/core sin necesidad de instalar dependencias adicionales. Devuelve el tipo de conexión disponible (ninguna, WiFi, datos móviles) lo que permite implementar el modo offline de forma precisa y mostrar mensajes adecuados al usuario según su estado de conectividad.

9. Webpack 5 via @nativescript/webpack

Se utilizó como bundler del proyecto. Es la configuración oficial del ecosistema NativeScript e incluye tree-shaking automático para reducir el tamaño del APK eliminando código no utilizado, soporte para TypeScript y módulos npm, y Hot Module Replacement durante el desarrollo que permite ver cambios sin reconstruir la aplicación completa.

10. Stack Backend: ASP.NET Core 8 con SQL Server

El cliente móvil se integra con el backend desarrollado por el equipo: una API REST construida con ASP.NET Core 8 en C# con autenticación JWT Bearer y SQL Server como base de datos relacional. La compatibilidad entre NativeScript y esta API se logra mediante el módulo Http de NativeScript Core que permite enviar cabeceras HTTP personalizadas, incluyendo el token JWT en el header Authorization de cada solicitud protegida.